

Deimos Gym · Suscripciones, Pagos y Factura

Informe técnico y funcional · 9 de octubre de 2026

1. Funcionamiento y relaciones

Suscripciones abre la ficha del socio. En un solo formulario se elige plan, estado, período, fecha y medio de pago. Confirmar pago valida y guarda la membresía y el cobro juntos. Si algo falla, se revierte la operación. Una cuota pendiente conserva su importe original.

Relación	Función
Usuario → Person	Perfil: nombre, apellido y DNI.
Usuario → Membership → Plan	Membresía, estado y precio del plan.
Usuario → MonthlyPayment	Una cuota por usuario y mes.
MonthlyPayment → PaymentMovement	Cobros y anulaciones de esa cuota.
PaymentMovement → reversal_of	La anulación enlaza al cobro original.

2. Medios de pago, cobertura y factura

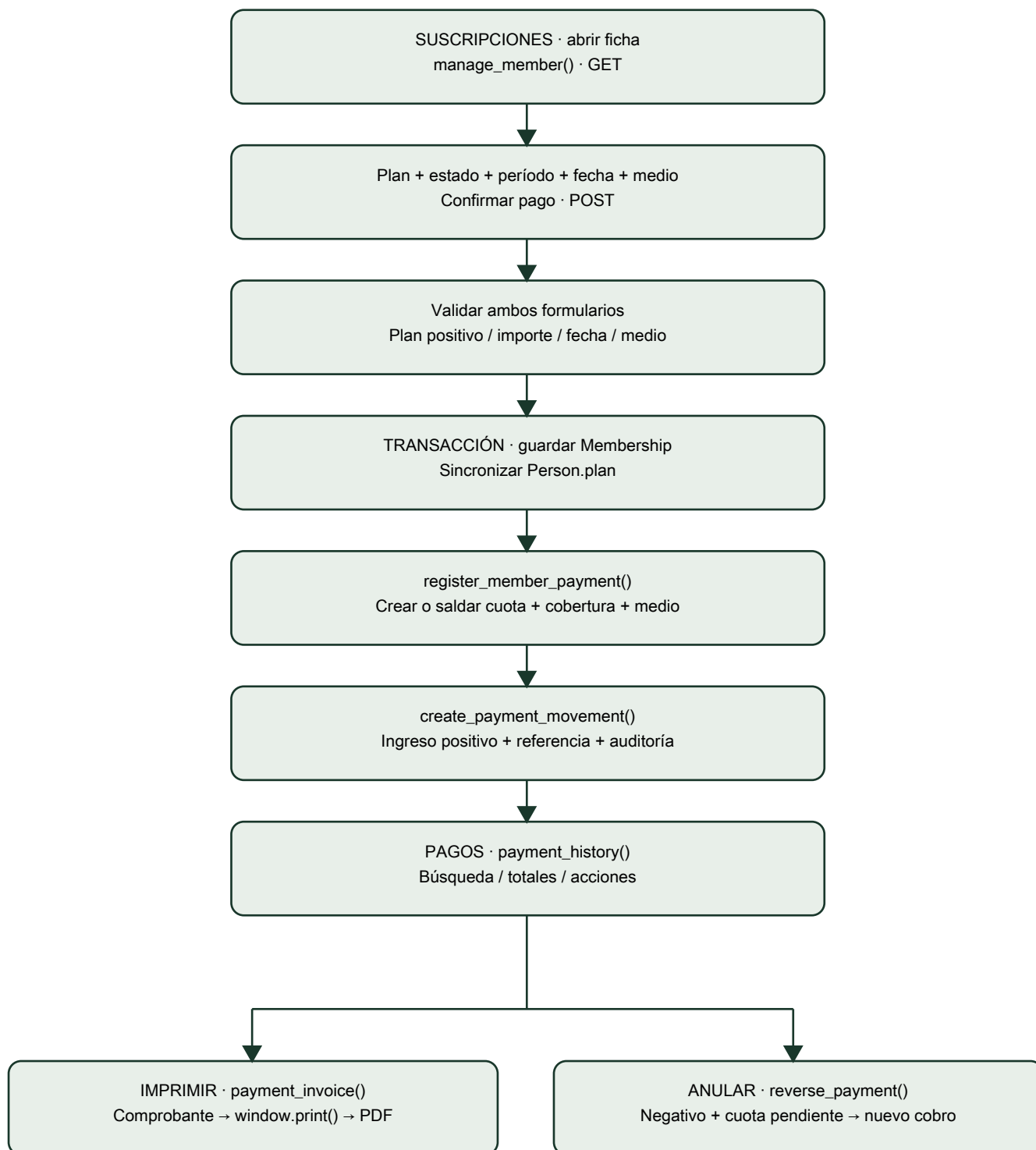
El medio es obligatorio: Efectivo (cash), Transferencia (transfer) o Tarjeta (card). Se almacena en method de la cuota y del movimiento; el historial y la factura muestran su etiqueta. La anulación conserva el medio original. Los pagos históricos sin ese dato muestran Sin registrar. La migración 0006_payment_method agrega los campos.

La cobertura comienza en la fecha de pago y vence el mismo día del mes siguiente, ajustado al último día si corresponde. La vigencia efectiva exige membresía activa y pago vigente; el vencimiento es inclusivo.

Pagos permite buscar por DNI o nombre y calcula ingresos, anulaciones y neto. ID MOVI. identifica el movimiento; ID FACT es la referencia automática PAGO-00000003, por ejemplo. Anular mantiene el cobro positivo y agrega uno negativo; la cuota queda pendiente y permite otro cobro.

La factura es una vista del movimiento, no un modelo independiente. Conserva su importe y medio, pero muestra el plan actual del socio: no se almacena el nombre histórico del plan. Es un comprobante interno sin validez fiscal. Los socios acceden solo a sus propios registros.

3. Gráfico del flujo



Si la validación falla, se muestran errores sin guardar. Si falla el cobro o la auditoría durante la transacción, se revierten todos los cambios. Imprimir no crea movimientos ni modifica el pago.

4. Bloques de código y ubicación de cada operación

Los bloques siguientes se extraen del código actual. Las rutas son relativas a crud_auth_original. Cada línea conserva su número para localizar la operación; estos números pueden variar con futuras ediciones.

4.1. manage_member

Ubicación: memberships/management_views.py · líneas 88–173

Recibe el formulario único; valida plan y pago. El bloque transaction.atomic() guarda Membership, limpia las relaciones en caché, llama al servicio del cobro y audita. El GET no cobra.

```
88 @login_required
89 @superuser_required
90 @require_http_methods(["GET", "POST"])
91 def manage_member(request, user_id):
92     user = get_object_or_404(User, pk=user_id)
93     person = Person.objects.select_related("plan").filter(user=user).first()
94     membership = (
95         Membership.objects.select_related("plan").filter(user=user).first()
96     )
97     if membership is None:
98         membership = Membership(user=user, plan=person.plan if person else None)
99     elif membership.plan_id is None and person and person.plan_id:
100         membership.plan = person.plan
101     today = timezone.localtime()
102     period = today.replace(day=1)
103     pending = MonthlyPayment.objects.filter(
104         member=user, period=period, paid_on__isnull=True
105     ).first()
106     initial = {
107         "period": period,
108         "paid_on": today,
109         "amount": (
110             pending.amount
111             if pending
112             else (membership.plan.base_price if membership.plan else None)
113         ),
114     }
115     data = request.POST if request.method == "POST" else None
116     membership_form = MemberManagementForm(data, instance=membership)
117     membership_valid = membership_form.is_valid() if data is not None else False
118     selected_plan = (
119         membership_form.cleaned_data.get("plan")
120         if membership_valid else membership.plan
121     )
122     payment_form = MemberPaymentForm(
123         data, member=user, initial=initial, selected_plan=selected_plan,
124     )
125     payment_valid = payment_form.is_valid() if data is not None else False
126     if membership_valid and payment_valid:
127         try:
128             with transaction.atomic():
129                 User.objects.select_for_update().get(pk=user.pk)
130                 saved = membership_form.save()
131                 user._state.fields_cache.pop("membership", None)
132                 user._state.fields_cache.pop("person", None)
133                 payment = register_member_payment(
134                     user, payment_form.cleaned_data, request.user,
135                 )
136                 audit(
137                     request, "UPDATE",
138                     f"Membresía {saved.pk}, plan {saved.plan_id}; "
139                     f"pago {payment.pk}: {payment.amount}",
140                 )
141         except ValidationError as error:
142             payment_form.add_error(None, error)
```

```

143         else:
144             messages.success(
145                 request,
146                 f"Suscripción y pago registrados. ID FACT: {payment.reference}.",
147             )
148             return redirect("manage_member", user_id=user.pk)
149 from checkin.services import member_checkin_status
150
151 status = member_checkin_status(person)
152 plan_prices = {
153     str(plan.pk): str(plan.base_price or 0) for plan in Plan.objects.all()
154 }
155 return render(
156     request,
157     "memberships/manage_member.html",
158     {
159         "member": user,
160         "person": person,
161         "membership": membership,
162         "membership_form": membership_form,
163
164         "payment_form": payment_form,
165         "membership_status": membership.effective_status,
166         "has_plans": Plan.objects.exists(),
167         "payment_amount_locked": bool(pending),
168         "plan_prices": plan_prices,
169         "payments": MonthlyPayment.objects.filter(
170             member=user
171         ).select_related("recorded_by")[:24],
172         "monthly_status": status,
173     },
174 )

```

4.2. MemberManagementForm

Ubicación: memberships/forms.py · líneas 5–26

Obliga a elegir plan y valida que tenga precio positivo. El estado es administrativo; no reemplaza la validación de cobertura.

```

5 class MemberManagementForm(forms.ModelForm):
6     status = forms.ChoiceField(
7         choices=[("active", "Activa"), ("inactive", "Inactiva")],
8         label="Estado de membresía",
9     )
10
11     class Meta:
12         model = Membership
13         fields = ["plan", "status"]
14         labels = {"plan": "Plan asignado"}
15
16     def __init__(self, *args, **kwargs):
17         super().__init__(*args, **kwargs)
18         for field in self.fields.values():
19             field.widget.attrs["class"] = "module-input"
20         self.fields["plan"].required = True
21
22     def clean_plan(self):
23         plan = self.cleaned_data["plan"]
24         if not plan.base_price or plan.base_price <= 0:
25             raise forms.ValidationError("Seleccioná un plan con precio positivo.")
26         return plan

```

4.3. MemberPaymentForm

Ubicación: payments/forms.py · líneas 22–100

Exige medio de pago y normaliza el período. Valida importe del plan o cuota pendiente y rechaza meses ya pagados. El atributo readonly es visual; las validaciones del servidor impiden alterar el importe.

```

22 class MemberPaymentForm(RecordPaymentForm):
23     method = forms.ChoiceField(
24         label="Medio de pago",
25         choices=[("", "Seleccioná un medio de pago"), *PAYMENT_METHODS],
26         widget=forms.Select(attrs={"class": "module-input"}),
27     )
28     period = forms.DateField(
29         label="Mes del pago",
30         input_formats=["%Y-%m"],
31         widget=forms.DateInput(
32             format="%Y-%m", attrs={"type": "month", "class": "module-input"}
33         ),
34     )
35
36     class Meta(RecordPaymentForm.Meta):
37         fields = ["period", "amount", "paid_on", "method"]
38         labels = {"amount": "Importe del pago"}
39         widgets = {
40             "amount": forms.NumberInput(
41                 attrs={"step": "0.01", "min": "0.01", "class": "module-input"}
42             )
43         }
44
45     def __init__(self, *args, member, selected_plan=None, **kwargs):
46         self.member = member
47
48         self.selected_plan = selected_plan
49         super().__init__(*args, **kwargs)
50         self.fields["amount"].widget.attrs["readonly"] = True
51         self.fields["amount"].help_text = (
52             "Se toma del plan seleccionado. Las cuotas pendientes conservan "
53             "su importe original."
54         )
55
56     def clean_period(self):
57         return self.cleaned_data["period"].replace(day=1)
58
59     def clean(self):
60         data = super().clean()
61         if data.get("period"):
62             existing = MonthlyPayment.objects.filter(
63                 member=self.member, period=data["period"]
64             ).first()
65             if existing and existing.paid_on:
66                 self.add_error("period", "Esta mensualidad ya está pagada")
67             elif (
68                 existing
69                 and data.get("amount") is not None
70                 and existing.amount != data["amount"]
71             ):
72                 self.add_error(
73                     "amount",
74                     (
75                         f"La cuota pendiente tiene un importe de ${existing.amount}. Ingresá "
76                         f"ese importe para saldarla"
77                     ),
78                 )
79             elif not existing:
80                 from .services import member_plan_amount
81
82                 try:
83                     amount = (
84                         self.selected_plan.base_price
85                         if self.selected_plan else member_plan_amount(self.member)
86                     )
87                 except ValidationError as error:
88                     self.add_error("amount", error)
89                 else:
90                     if (
91                         data.get("amount") is not None
92                         and data["amount"] != amount
93                     ):

```

```

92         ):
93             self.add_error(
94                 "amount",
95                 (
96                     f"El importe debe coincidir con el precio del plan seleccionado: $"
97
98                     f"{amount}."
99                 ),
100             )
101     return data

```

4.4. register_member_payment

Ubicación: payments/services.py · líneas 32–75

Obtiene o crea MonthlyPayment y bloquea la cuota. Guarda fecha, medio y operador, aplica cobertura y llama al creador del movimiento. La función devuelve la cuota; el movimiento se crea por separado.

```

32 @transaction.atomic
33 def register_member_payment(member, data, operator):
34     """Settle an existing charge or create one, preserving its recorded price."""
35     period = data["period"]
36     existing = MonthlyPayment.objects.filter(
37         member=member, period=period
38     ).first()
39     amount = existing.amount if existing else member_plan_amount(member)
40     if data["amount"] != amount:
41         raise ValidationError(
42             (
43                 f"El importe del pago debe ser ${amount}, según la cuota pendiente o "
44                 f"el plan seleccionado."
45             )
46         )
47     payment, _ = MonthlyPayment.objects.get_or_create(
48         member=member,
49         period=period,
50         defaults={
51             "amount": amount,
52             "due_date": period.replace(
53                 day=min(
54                     data["paid_on"].day,
55                     monthrange(period.year, period.month)[1],
56                 )
57         ),
58     },
59 )
60 payment = MonthlyPayment.objects.select_for_update().get(pk=payment.pk)
61 if payment.paid_on:
62     raise ValidationError("Esta mensualidad ya está pagada")
63 if payment.amount != data["amount"]:
64     raise ValidationError(
65         f"El importe de la cuota pendiente es ${payment.amount}"
66     )
67 payment.paid_on = data["paid_on"]
68 payment.method = data.get("method", "")
69 payment.reference = ""
70 payment.recorded_by = operator
71 apply_payment_coverage(payment)
72 payment.full_clean()
73 payment.save()
74 create_payment_movement(payment)
75 return payment

```

4.5. create_payment_movement

Ubicación: payments/services.py · líneas 116–130

Aquí se crea el INGRESO positivo. Copia importe, fecha, operador y medio de la cuota a PaymentMovement. Genera la referencia con el ID del movimiento y la guarda también en la cuota.

```
116 @transaction.atomic
117 def create_payment_movement(payment):
118     """Congela los datos del cobro para que futuras modificaciones no alteren el historial."""
119     movement = PaymentMovement.objects.create(
120         charge=payment,
121         amount=payment.amount,
122         paid_on=payment.paid_on,
123         recorded_by=payment.recorded_by,
124         method=payment.method,
125     )
126     movement.reference = f"PAGO-{movement.pk:08d}"
127     movement.save(update_fields=["reference"])
128     payment.reference = movement.reference
129     payment.save(update_fields=["reference"])
130     return movement
```

4.6. apply_payment_coverage

Ubicación: payments/services.py · líneas 12–16

Define el inicio y fin desde la fecha real de pago; utiliza next_month_expiry() para ajustar meses con distinta cantidad de días.

```
12 def apply_payment_coverage(payment):
13     payment.coverage_start = payment.paid_on
14     payment.coverage_end = (
15         next_month_expiry(payment.paid_on) if payment.paid_on else None
16     )
```

4.7. membership_coverage

Ubicación: memberships/services.py · líneas 26–61

Combina estado administrativo con último pago y vencimiento. Al anular un pago puede quedar disponible la cobertura de uno anterior.

```
26 def membership_coverage(user, today=None, membership=None):
27     """Shared coverage rule: manual activation plus payment, inclusive expiry."""
28     today = today or timezone.localtime()
29     membership = (
30         membership
31         if membership is not None
32         else getattr(user, "membership", None)
33     )
34     if (
35         membership is not None
36         and hasattr(membership, "last_paid_on")
37         and today == timezone.localtime()
38     ):
39         paid_on = membership.last_paid_on
40         coverage_end = getattr(membership, "last_coverage_end", None)
41     else:
42         payment = (
43             user.monthly_payments.filter(paid_on__lte=today)
44             .order_by("-paid_on", "-period", "-pk")
45             .first()
46         )
47         paid_on = payment.paid_on if payment else None
48         coverage_end = payment.coverage_end if payment else None
49     expiry = (coverage_end or next_month_expiry(paid_on)) if paid_on else None
50     days_remaining = (expiry - today).days if expiry else None
51
52     administratively_active = (
53         membership is not None and membership.status == "active"
54     )
```

```

54     active = administratively_active and expiry is not None and expiry >= today
55     return {
56         "active": active,
57         "administratively_active": administratively_active,
58         "last_paid_on": paid_on,
59         "expires_at": expiry,
60         "days_remaining": days_remaining,
61     }

```

4.8. payment_history

Ubicación: payments/views.py · líneas 96–147

Consulta movimientos respetando permisos, aplica búsqueda por socio y calcula ingresos, egresos y neto. Los filtros internos por ID sirven a los enlaces de cobro/anulación, no al buscador visible.

```

96 @login_required
97 @require_http_methods(["GET"])
98 def payment_history(request):
99     movements = PaymentMovement.objects.select_related(
100         "charge__member__person", "recorded_by", "reversal_of", "reversal"
101     )
102     if not request.user.has_perm("pagos.view_monthlypayment"):
103         movements = movements.filter(charge__member=request.user)
104     search = request.GET.get("q", "").strip()
105     if search:
106         if search.isascii() and search.isdigit():
107             movements = (
108                 movements.filter(charge__member__person__id_number=int(search))
109                 if len(search) <= 10 else movements.none()
110             )
111         else:
112             for word in search.split():
113                 movements = movements.filter(
114                     Q(charge__member__person__name__icontains=word)
115                     | Q(charge__member__person__surname__icontains=word)
116                     | Q(charge__member__first_name__icontains=word)
117                     | Q(charge__member__last_name__icontains=word)
118                     | Q(charge__member__username__icontains=word)
119                 )
120     payment_id = request.GET.get("payment_id", "").strip()
121
122     member_id = request.GET.get("member_id", "").strip()
123     for value, field in [(payment_id, "pk"), (member_id, "charge__member_id")]:
124         if value:
125             movements = (
126                 movements.filter(**{field: int(value)})
127                 if value.isascii() and value.isdigit() and len(value) <= 18
128                 else movements.none()
129             )
130     totals = movements.aggregate(
131         ingresos=Sum("amount", filter=Q(amount__gt=0)),
132         egresos=Sum("amount", filter=Q(amount__lt=0)),
133         ingresos_cantidad=Count("pk", filter=Q(amount__gt=0)),
134         egresos_cantidad=Count("pk", filter=Q(amount__lt=0)),
135         neto=Sum("amount"),
136     )
137     context = {
138         "movements": movements,
139         "search": search,
140         "payment_id": payment_id,
141         "member_id": member_id,
142         "ingresos": totals["ingresos"] or Decimal("0"),
143         "egresos": -(totals["egresos"] or Decimal("0")),
144         "ingresos_cantidad": totals["ingresos_cantidad"],
145         "egresos_cantidad": totals["egresos_cantidad"],
146         "total": totals["neto"] or Decimal("0"),
147     }
148     return render(request, "payments/history.html", context)

```

4.9. cancel_payment

Ubicación: payments/views.py · líneas 182–208

GET muestra la confirmación. POST ejecuta reverse_payment() y audita; requiere permisos de consulta y modificación de pagos.

```
182 @login_required
183 @permission_required(
184     ["pagos.change_monthlypayment", "pagos.view_monthlypayment"],
185     raise_exception=True,
186 )
187 @require_http_methods(["GET", "POST"])
188 def cancel_payment(request, movement_id):
189     movement = get_object_or_404(
190         PaymentMovement.objects.select_related("charge__member"), pk=movement_id
191     )
192     if request.method == "POST":
193         try:
194             with transaction.atomic():
195                 reversal = reverse_payment(movement.pk, request.user)
196                 audit(
197                     request,
198                     "CREATE",
199                     f"Anulación {reversal.pk} del pago {movement.pk}: {reversal.amount}",
200                 )
201         except ValidationError as error:
202             messages.error(request, "; ".join(error.messages))
203         else:
204             messages.success(
205                 request, "Pago anulado mediante un movimiento negativo."
206             )
207
208     return redirect("payment_history")
209     return render(request, "payments/cancel.html", {"movement": movement})
```

4.10. reverse_payment

Ubicación: payments/services.py · líneas 133–175

Aquí se crea la ANULACIÓN negativa. reversal_of enlaza al original; conserva cuota, referencia y medio. Limpia fecha y cobertura de la cuota para habilitar un nuevo pago. Impide anulaciones repetidas.

```
133 @transaction.atomic
134 def reverse_payment(movement_id, operator):
135     from django.utils import timezone
136
137     # Bloquear la cuota serializa tanto nuevos cobros como anulaciones.
138     original = PaymentMovement.objects.get(pk=movement_id)
139     charge = MonthlyPayment.objects.select_for_update().get(
140         pk=original.charge_id
141     )
142     original = PaymentMovement.objects.select_for_update().get(pk=movement_id)
143     if (
144         original.amount <= 0
145         or PaymentMovement.objects.filter(reversal_of=original).exists()
146     ):
147         raise ValidationError(
148             "Este movimiento ya fue anulado o es una anulación."
149         )
150     reversal = PaymentMovement(
151         charge=charge,
152         amount=-original.amount,
153         paid_on=timezone.localtime(),
154         reference=original.reference,
155         method=original.method,
156         reversal_of=original,
157         recorded_by=operator,
```

```

158     )
159     reversal.full_clean()
160     reversal.save()
161     charge.paid_on = None
162     charge.reference = ""
163     charge.coverage_start = None
164     charge.coverage_end = None
165     charge.recorded_by = None
166     charge.save(
167         update_fields=[
168             "paid_on",
169             "reference",
170             "recorded_by",
171             "coverage_start",
172             "coverage_end",
173         ]
174     )
175     return reversal

```

4.11. payment_invoice

Ubicación: payments/views.py · líneas 150–179

Aquí se consulta el COMPROBANTE. Reúne movimiento, cuota, usuario, perfil y membresía; verifica acceso y renderiza invoice.html. No guarda una factura ni genera otro movimiento.

```

150 @login_required
151 @require_http_methods(["GET"])
152 def payment_invoice(request, movement_id):
153     movements = PaymentMovement.objects.select_related(
154         "charge__member__person__plan",
155         "charge__member__membership__plan",
156         "recorded_by",
157         "reversal_of",
158         "reversal",
159     )
160     if not request.user.has_perm("pagos.view_monthlypayment"):
161         movements = movements.filter(charge__member=request.user)
162     movement = get_object_or_404(movements, pk=movement_id)
163     member = movement.charge.member
164     person = getattr(member, "person", None)
165     membership = getattr(member, "membership", None)
166     plan = (membership.plan if membership else None) or (
167         person.plan if person else None
168     )
169     return render(
170         request,
171         "payments/invoice.html",
172         {
173             "movement": movement,
174             "member": member,
175             "person": person,
176             "membership": membership,
177             "plan": plan,
178         },
179     )

```

4.12. sync_membership_plan

Ubicación: accounts/signals.py · líneas 53–83

Al guardar Membership sincroniza el plan en Person. La señal queda incluida en la transacción que confirma el pago.

```

53 @receiver(post_save, sender=Membership)
54 def sync_membership_plan(
55     sender,
56     instance,

```

```

57     created=False,
58     raw=False,
59     update_fields=None,
60     using=None,
61     **kwargs
62 ):
63     if raw or (
64         update_fields is not None
65         and "plan" not in update_fields
66         and "plan_id" not in update_fields
67     ):
68         return
69     if created and instance.plan_id is None:
70         plan_id = (
71             Person.objects.using(using)
72             .filter(user_id=instance.user_id)
73             .values_list("plan_id", flat=True)
74             .first()
75         )
76         if plan_id is not None:
77             Membership.objects.using(using).filter(pk=instance.pk).update(
78
79                 plan_id=plan_id
80             )
81             instance.plan_id = plan_id
82             Person.objects.using(using).filter(user_id=instance.user_id).update(
83                 plan_id=instance.plan_id
84             )

```

4.13. activate_paid_membership

Ubicación: accounts/signals.py · líneas 119–134

Al guardar una cuota pagada obtiene o crea la membresía si falta. No cambia el estado manual de una membresía existente.

```

119 @receiver(post_save, sender=MonthlyPayment)
120 def activate_paid_membership(sender, instance, raw=False, using=None, **kwargs):
121     if raw or not instance.paid_on or instance.paid_on > timezone.localtime():
122         return
123     if next_month_expiry(instance.paid_on) < timezone.localtime():
124         return
125     plan_id = (
126         Person.objects.using(using)
127         .filter(user_id=instance.member_id)
128         .values_list("plan_id", flat=True)
129         .first()
130     )
131     membership, _ = Membership.objects.using(using).get_or_create(
132         user_id=instance.member_id,
133         defaults={"plan_id": plan_id, "status": "active"},
134     )

```

5. Interfaz y almacenamiento

memberships/templates/memberships/manage_member.html: formulario POST con CSRF y un único botón Confirmar pago.

staticfiles/js/member-payment.js: actualiza el importe visible al seleccionar plan. La validación definitiva está en Python.

payments/templates/payments/history.html: presenta cada movimiento con DNI, medio, referencia, estado y acciones.

payments/templates/payments/invoice.html: presenta la factura. payment-invoice.css define el estilo y payment-invoice.js ejecuta window.print(). Ambos están en staticfiles/style y staticfiles/js.

payments/models.py: define cuota, movimientos, relaciones y PAYMENT_METHODS.

payments/migrations/0006_payment_method.py agrega method a los dos modelos sin inventar medios para pagos anteriores.